BEYOND / TAHIR

PRACTICAL SETUP PLAYBOOK

# FREE AI GATEWAY SETUP GUIDE

Connect **FreeLLMAPI** and **OmniRoute** with Claude Code, Codex and OpenClaw - one clear tutorial at a time.

CLAUDE CODE | CODEX | OPENCLAW

**Inside**
Install, configure, test, troubleshoot and secure all six integrations.

**FreeLLMAPI**
GitHub repository

**OmniRoute**
GitHub repository

START HERE

# How to Use This Guide

**This guide is designed to be followed in order, but every integration also works as a standalone tutorial.**


| | | |
|---|---|---|
| 01 | **Understand the setup**<br>Architecture and what the gateways actually do | 03 |
| 02 | **Before you start**<br>Clients, prerequisites and provider access | 04 |
| 03 | **URL cheat sheet**<br>The root URL versus the /v1 URL | 05 |
| 04 | **FreeLLMAPI**<br>Install, dashboard, Claude, Codex and OpenClaw | 06 |
| 05 | **OmniRoute**<br>Install, dashboard, Claude, Codex and OpenClaw | 14 |
| 06 | **Which one should you use?**<br>Practical comparison and recommendation | 20 |
| 07 | **Troubleshooting**<br>URLs, authentication, clients and Docker | 21 |
| 08 | **Security and final checklist**<br>Use free gateways without being careless | 23 |


READ THIS FIRST

### Free does not mean keyless.

You still need legitimate provider API keys, OAuth access or provider free tiers. These projects organize and route that access; they do not unlock paid models or bypass restrictions.

LINKS

### Built for screen and print.

Every blue or red link is clickable in the digital PDF. QR codes are included for the two repositories and the BeyondTahir website.

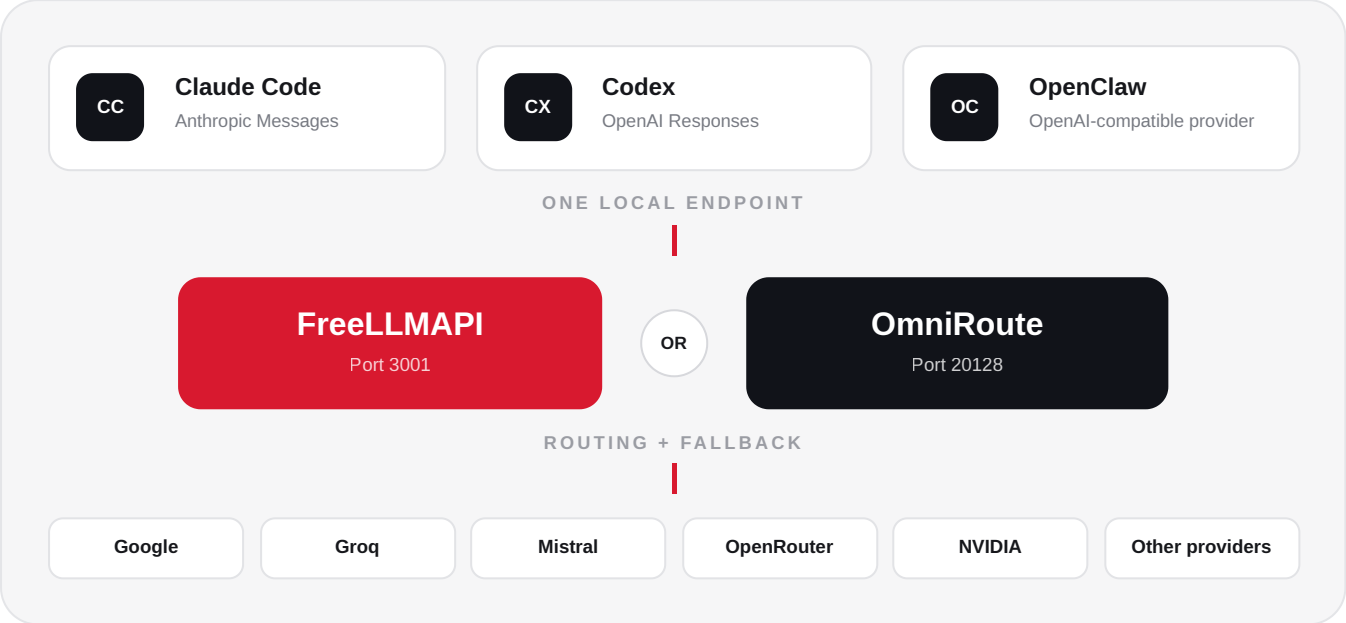
TERMINOLOGY

### Claude means Claude Code.

This guide focuses on the local coding client, not the claude.ai website. Codex means the Codex CLI or IDE extension. OpenClaw means the locally configured agent gateway.

HOW IT WORKS

# The Setup in One Picture




## What the gateways do

- **Unify access:** your tools connect to one local base URL and one gateway key.
- **Route requests:** choose a model, use an automatic route or create a fallback chain.
- **Track limits:** manage multiple provider connections without editing every client.
- **Translate protocols:** expose the wire format expected by supported coding tools.

## What they do not do

- They do not make every paid model free.
- They do not remove provider rate limits, regional rules or terms of service.
- They do not guarantee that every chat model can perform coding-tool calls.
- They do not provide a production SLA simply because the router is running.

**THE USEFUL MENTAL MODEL**

The gateway is a smart switchboard. Your AI tool speaks to the switchboard; the switchboard decides which connected provider receives the request.

PREREQUISITES

# Before You Start

Install only the clients you plan to use. The gateway tutorials later in this guide assume the client already launches successfully on your computer.

CC

## Claude Code

Official native installer for macOS, Linux and WSL.

MACOS / LINUX / WSL

```
curl -fsSL
https://claude.ai/install
.sh | bash
```

Official quickstart ->

CX

## Codex

Install the OpenAI Codex CLI with npm.

TERMINAL

```
npm install -g
@openai/codex
```

Official Codex CLI guide ->

OC

## OpenClaw

Recommended installer for macOS, Linux and WSL2.

MACOS / LINUX / WSL2

```
curl -fsSL
https://openclaw.ai/insta
ll.sh | bash
```

Official install guide ->

## You also need

- [ ] Node.js 20+ for the setup CLIs and OmniRoute.
- [ ] Docker only when using the FreeLLMAPI Docker path.
- [ ] At least one legitimate provider connection or free-tier key.
- [ ] A terminal and permission to edit your own user configuration files.

## Verify the clients first

RUN WHAT YOU INSTALLED

```
claude --version
codex --version
openclaw --version
```

Fix a client installation problem before connecting it to a router. Otherwise two different problems get mixed together.

**WINDOWS USERS**

Use the official PowerShell installer shown in each project's linked documentation, or use WSL2 where recommended. The router configuration concepts remain the same.

SAVE THIS PAGE

# The URL Cheat Sheet

Most failed setups are not model problems. They are one missing or duplicated **/v1**.

| Where it is used | FreeLLMAPI | OmniRoute |
|---|---|---|
| **Dashboard**<br>Open in a browser | http://localhost:3001 | http://localhost:20128 |
| **Claude Code**<br>Gateway root | `http://localhost:3001` | `http://localhost:20128` |
| **Codex**<br>OpenAI-compatible API | `http://localhost:3001/v1` | `http://localhost:20128/v1` |
| **OpenClaw**<br>Custom provider base URL | `http://127.0.0.1:3001/v1` | `http://127.0.0.1:20128/v1` |

## Why Claude uses the root

Claude Code appends the Anthropic Messages path itself. Give it the server root - without `/v1`.

CORRECT

```
ANTHROPIC_BASE_URL=http://localhost:3001
```

## Why Codex and OpenClaw use /v1

They connect to an OpenAI-compatible API surface. Their configured base URL includes `/v1`.

CORRECT

```
base_url = "http://localhost:3001/v1"
```

**FreeLLMAPI health test**

TEST

```
curl http://localhost:3001/v1/models \
  -H "Authorization: Bearer YOUR_KEY"
```

**OmniRoute health test**

TEST

```
curl http://localhost:20128/v1/models \
  -H "Authorization: Bearer YOUR_KEY"
```

FAST DIAGNOSIS

A 404 often means the wrong path. A 401 usually means the wrong gateway key. Connection refused usually means the router is not running or the port is wrong.

PART ONE

FreeLLMAPI

01

LOCAL FREE-TIER ROUTER

# FreeLLMAPI

Best when the main goal is to pool legitimate free-tier access behind one simple OpenAI-compatible gateway.

## What you will configure

- Install the router
- Add provider keys
- Copy one unified key
- Connect Claude Code
- Connect Codex
- Connect OpenClaw

**Scan for the repository**
github.com/tashfeenahmed/freellmapi

**GitHub repository**
Source code, README and releases
https://github.com/tashfeenahmed/freellmapi ->

**Install and deploy**
Docker, desktop and local development
https://github.com/tashfeenahmed/freellmapi/blob/main/docs/install.md ->

**Clients and coding agents**
Claude Code, Codex and other clients
https://github.com/tashfeenahmed/freellmapi/blob/main/docs/clients/01-agent-clients.md ->

**Current model catalog**
Available models, providers and quotas
https://freellmapi.co/models.html ->

STEP 1 OF 3

# Install FreeLLMAPI

Choose either the desktop installer or Docker. Do not install both unless you deliberately want separate instances.

### Option A - Desktop app

Use the latest macOS or Windows installer from GitHub Releases.

**Open FreeLLMAPI releases**
Download the installer for your operating system
https://github.com/tashfeenahmed/freellmapi/releases/latest
->

After launch, the local dashboard and API are served on port 3001.

### Option B - Docker one-liner

Docker is required. The installer creates the working directory, generates an encryption key, pulls the image and starts the container.

TERMINAL

```
curl -fsSL
https://freellmapi.co/install.sh | bash
```

The generated encryption key protects provider credentials stored by the local gateway.

1 **Open the local dashboard**

Go to http://localhost:3001.

2 **Confirm the service is running**

QUICK CHECK

```
curl -I http://localhost:3001
```

3 **Keep the router local while learning**

Do not expose port 3001 publicly until authentication, networking and firewall rules are understood.

Expected local path

Your computer -> localhost:3001 -> FreeLLMAPI dashboard + API

DOCKER NETWORKING REMINDER

When another client also runs inside Docker, localhost means that client container. Use a shared Docker network, service name or a host-gateway address instead.

FREELLMAPI SETUP

# Connect Providers and Create One Key

1. Add provider access
2. Arrange fallback chain
3. Copy unified key
4. Test /v1/models

## 1 Add legitimate provider keys

Open the **Keys** page and add the free-tier or paid provider credentials you are authorized to use.

## 2 Set the fallback order

Move the models or providers you prefer to the top of the fallback chain. Keep more than one working route when reliability matters.

## 3 Copy the FreeLLMAPI unified key

This is the client-facing key used by Claude Code, Codex and OpenClaw. Do not paste the original Google, Groq or other upstream key into those clients.

MACOS / LINUX / WSL

```
export FREELLMAPI_API_KEY="replace-with-unified-key"
```

WINDOWS POWERSHELL

```
$env:FREELLMAPI_API_KEY="replace-with-unified-key"
```

## 4 Test the gateway before touching any client

TEST AVAILABLE MODELS

```
curl -s http://localhost:3001/v1/models \
  -H "Authorization: Bearer $FREELLMAPI_API_KEY"
```

**Upstream keys**
Stay inside FreeLLMAPI

->

**Unified gateway key**
Given to your local clients

INTERPRET THE RESULT

**Models returned:** the gateway is ready. **401:** check the unified key. **Empty list:** add or enable a provider. **Connection refused:** start FreeLLMAPI.

TUTORIAL 1 OF 3

# Use FreeLLMAPI with Claude Code

CC

## FreeLLMAPI -> Claude Code

Use the gateway root on port 3001. Claude Code expects the Anthropic Messages interface and adds its own path.

1

### Preview the generated configuration

SAFE PREVIEW

```
npx freellmapi setup-claude \
  --url http://localhost:3001 \
  --dry-run
```

2

### Apply the configuration

APPLY

```
npx freellmapi setup-claude \
  --url http://localhost:3001
```

3

### Launch Claude Code inside a project

START

```
cd /path/to/your/project
claude
```

4

### Verify before giving edit permission

Inside Claude Code, run `/status`. Confirm the base URL points to `http://localhost:3001`, then use a read-only test prompt.

TEST PROMPT

```
Examine this repository and explain what it does.
Do not modify any files.
```

### Temporary launcher

Keep the gateway credential out of Claude settings for that run.

NO-PERSISTENCE LAUNCH

```
npx freellmapi launch
```

### Manual one-session launch

ONE SESSION

```
ANTHROPIC_BASE_URL="http://localhost:3001" \
ANTHROPIC_AUTH_TOKEN="$FREELLMAPI_API_KEY" \
claude
```

DO NOT ADD /V1

For Claude Code, the correct base is the server root. A duplicated or incorrect path is one of the most common causes of 404 errors.

TUTORIAL 2 OF 3

# Use FreeLLMAPI with Codex

CX

## FreeLLMAPI -> Codex

Codex connects to the OpenAI-compatible `/v1` surface and uses the Responses API through the generated provider configuration.

1

### Preview what will change

SAFE PREVIEW

```
npx freellmapi setup-codex \
  --url http://localhost:3001 \
  --dry-run
```

2

### Apply the Codex configuration

APPLY

```
npx freellmapi setup-codex \
  --url http://localhost:3001
```

3

### Start Codex

START

```
cd /path/to/your/project
codex
```

4

### Run a read-only test

TEST PROMPT

```
Inspect this repository and identify the main entry point.
Do not edit anything.
```

### Temporary launcher

Launch Codex with temporary overrides and injected credentials.

NO-PERSISTENCE LAUNCH

```
npx freellmapi launch-codex
```

### Pin a tested model

List live IDs, then rerun setup with one exact ID.

FIXED MODEL

```
curl -s http://localhost:3001/v1/models \
  -H "Authorization: Bearer
$FREELLMAPI_API_KEY"

npx freellmapi setup-codex \
  --url http://localhost:3001 \
  --model "EXACT_MODEL_ID"
```

CONNECTIVITY IS NOT CAPABILITY

A model can answer normal chat and still fail at tool calls, patches or structured output. Test a complete coding loop before relying on it.

TUTORIAL 3 OF 3 - PART A

# Configure OpenClaw for FreeLLMAPI

OC

## FreeLLMAPI -> OpenClaw

FreeLLMAPI does not currently expose a dedicated `setup-openclaw` generator, so use OpenClaw's custom provider configuration.

### 1 Store the gateway key where the OpenClaw service can read it

CREDENTIAL FILE

```
mkdir -p ~/.openclaw
cat >> ~/.openclaw/.env <<'EOF'
FREELLMAPI_API_KEY=replace-with-unified-key
EOF
chmod 600 ~/.openclaw/.env
```

### 2 Back up the existing OpenClaw config

BACKUP

```
cp ~/.openclaw/openclaw.json \
   ~/.openclaw/openclaw.json.backup 2>/dev/null || true
```

### 3 Merge this provider into openclaw.json

Edit `~/.openclaw/openclaw.json`. Preserve unrelated agents, channels, memory and security settings.

OPENCLAW.JSON - JSON5

```
{
  agents: {
    defaults: {
      model: { primary: "freellmapi/auto" }
    }
  },
  models: {
    mode: "merge",
    providers: {
      freellmapi: {
        baseUrl: "http://127.0.0.1:3001/v1",
        apiKey: "${FREELLMAPI_API_KEY}",
        api: "openai-completions",
        models: [
          { id: "auto", name: "FreeLLMAPI Auto" }
        ]
      }
    }
  }
}
```

TUTORIAL 3 OF 3 - PART B

# Finish the FreeLLMAPI OpenClaw Setup

OC

## Validate and Test OpenClaw

Do not jump straight into a long autonomous task. Validate the config, confirm the active route and then send one simple prompt.

1

### Validate the JSON5 configuration

VALIDATE

```
openclaw config validate
```

2

### Select and inspect the model route

MODEL ROUTE

```
openclaw models set freellmapi/auto
openclaw models status
```

3

### Restart the background gateway

RESTART

```
openclaw gateway restart
```

4

### Open the TUI and run a simple test

START OPENCLAW

```
openclaw
```

TEST MESSAGE

```
Reply exactly: FREELLMAPI_OK
```

### Use a specific model

Fetch the exact ID from `/v1/models`, add that ID to the provider's `models` array, and change the primary route to `freellmapi/EXACT_MODEL_ID`.

### If the daemon misses the key

A variable exported in one terminal may not reach a background service. Keep the router credential in `~/.openclaw/.env`, then restart the gateway.

config validate -> models set -> gateway restart -> simple test

**USE AUTO CAREFULLY**

Automatic routing is convenient for learning. For repeatable coding work, choose an exact model that you have tested for tools, context length and reliability.

REALITY CHECK

# FreeLLMAPI: Limits You Should Expect

Free-tier aggregation is useful, but the trade-offs should be visible before you use it for important work.

### Top model slows or disappears

Daily or monthly free quotas can be consumed. Move another tested model into the fallback chain or wait for the provider reset.

### Chat works, agent fails

The chosen model may not support tool calling, structured responses or reliable patch generation. Pin a model that does.

### The model list is empty

No provider is active, a key is invalid or the route is disabled. Test each provider in the dashboard.

### Quality changes during the day

Automatic routing may fall from a preferred model to a weaker fallback after a quota is reached.

### Remote client cannot connect

FreeLLMAPI binds locally by default. Use a secure network design rather than exposing the port casually.

### Docker says localhost refused

Inside Docker, localhost is the current container. Use a shared network or host-gateway address.

PRODUCTION RULE

Keep at least one dependable paid or controlled provider as the final fallback for client work. Free is a pricing condition, not an infrastructure strategy.

**Read the project's own limitation notes:**
`github.com/tashfeenahmed/freellmapi`

PART TWO

OmniRoute

02

ADVANCED LOCAL AI GATEWAY

# OmniRoute

Best when you want broader provider management, routing combinations, telemetry and deeper client integration.

## What you will configure

- Install the global CLI
- Connect providers
- Create an endpoint key
- Build optional combos
- Connect Claude Code
- Connect Codex and OpenClaw

**Scan for the repository**
github.com/diegosouzapw/OmniRoute

**GitHub repository**
Source code, README and releases
https://github.com/diegosouzapw/OmniRoute ->

**Setup guide**
Install methods and CLI configuration
https://github.com/diegosouzapw/OmniRoute/blob/release/v3.8.51/docs/guides/SETUP_GUIDE.md ->

**CLI tools reference**
Claude Code, Codex, OpenClaw and more
https://github.com/diegosouzapw/OmniRoute/blob/release/v3.8.51/docs/reference/CLI-TOOLS.md ->

**Codex configuration guide**
Responses API and TOML examples
https://github.com/diegosouzapw/OmniRoute/blob/release/v3.8.51/docs/guides/CODEX-CLI-CONFIGURATION.md ->

OMNIROUTE SETUP

# Install and Prepare OmniRoute

## 1 Install the OmniRoute CLI

INSTALL

```
npm install -g omniroute
```

## 2 Start the local server

START

```
omniroute
```

DASHBOARD

```
http://localhost:20128
```

API BASE

```
http://localhost:20128/v1
```

## 3 Connect at least one provider

Open **Providers**, add an API-key or OAuth connection supported by OmniRoute, and run its connection test.

## 4 Create a client-facing endpoint key

Open **Endpoints** or **API Manager**, create a key, and copy it when displayed. This is the key your local tools use.

## 5 Optionally create a Combo

Use **Combos** to define a preferred model and fallbacks. Start simple; complexity before testing is just decorative confusion.

MACOS / LINUX / WSL

```
export OMNIROUTE_API_KEY="replace-with-endpoint-key"
```

WINDOWS POWERSHELL

```
$env:OMNIROUTE_API_KEY="replace-with-endpoint-key"
```

## 6 Test the endpoint

TEST MODELS

```
curl -s http://localhost:20128/v1/models \
  -H "Authorization: Bearer $OMNIROUTE_API_KEY"
```

KEY DISTINCTION

The OmniRoute endpoint key is for clients to access OmniRoute. It is not the same as an upstream provider key stored inside the router.

TUTORIAL 1 OF 3

# Use OmniRoute with Claude Code

CC

## OmniRoute -> Claude Code

The generator reads the live catalog and writes the Claude Code configuration expected on your machine.

### 1 Preview the generated changes

SAFE PREVIEW

```
omniroute setup-claude \
  --api-key "$OMNIROUTE_API_KEY" \
  --dry-run
```

### 2 Apply the configuration

APPLY

```
omniroute setup-claude \
  --api-key "$OMNIROUTE_API_KEY"
```

### 3 Launch and verify

START

```
cd /path/to/your/project
claude
```

Run `/status` inside Claude Code. The base URL should be `http://localhost:20128` without `/v1`.

### 4 Use a read-only test prompt

TEST PROMPT

```
Read this project and explain its architecture.
Do not make any changes.
```

### Temporary launcher

INJECT FOR THIS RUN

```
omniroute run claude --model auto
```

The legacy `omniroute launch` command is also retained.

### Manual one-session launch

ONE SESSION

```
ANTHROPIC_BASE_URL="http://127.0.0.1:20128" \
ANTHROPIC_AUTH_TOKEN="$OMNIROUTE_API_KEY" \
claude
```

**CHECK BOTH ROUTING AND CREDENTIAL SOURCE**

A custom base URL without a gateway credential can leave an existing Claude login active. The `/status` screen is your first diagnostic.

TUTORIAL 2 OF 3

# Use OmniRoute with Codex

CX

## OmniRoute -> Codex

Use the generated profile when possible. The manual TOML below is included for transparency and troubleshooting.

### 1 Preview and apply

PREVIEW, THEN APPLY

```
omniroute setup-codex \
  --api-key "$OMNIROUTE_API_KEY" \
  --dry-run

omniroute setup-codex \
  --api-key "$OMNIROUTE_API_KEY"
```

### 2 Start Codex

START

```
cd /path/to/your/project
codex
```

### 3 Manual provider configuration when needed

Edit `~/.codex/config.toml`. Root keys must appear before the provider table.

CODEX CONFIG.TOML

```
model = "auto"
model_provider = "omniroute"

[model_providers.omniroute]
name                = "OmniRoute"
base_url            = "http://127.0.0.1:20128/v1"
env_key             = "OMNIROUTE_API_KEY"
requires_openai_auth = false
wire_api            = "responses"
```

### Temporary launch

ONE RUN

```
omniroute run codex --model auto
```

The legacy `omniroute launch-codex` command also remains available.

### Predictable output

Replace `model = "auto"` with an exact live model ID after testing it for tools, patches and context.

MODERN CODEX CONFIGURATION

User-level settings live in `~/.codex/config.toml`. Old YAML configuration from legacy clients may be ignored.

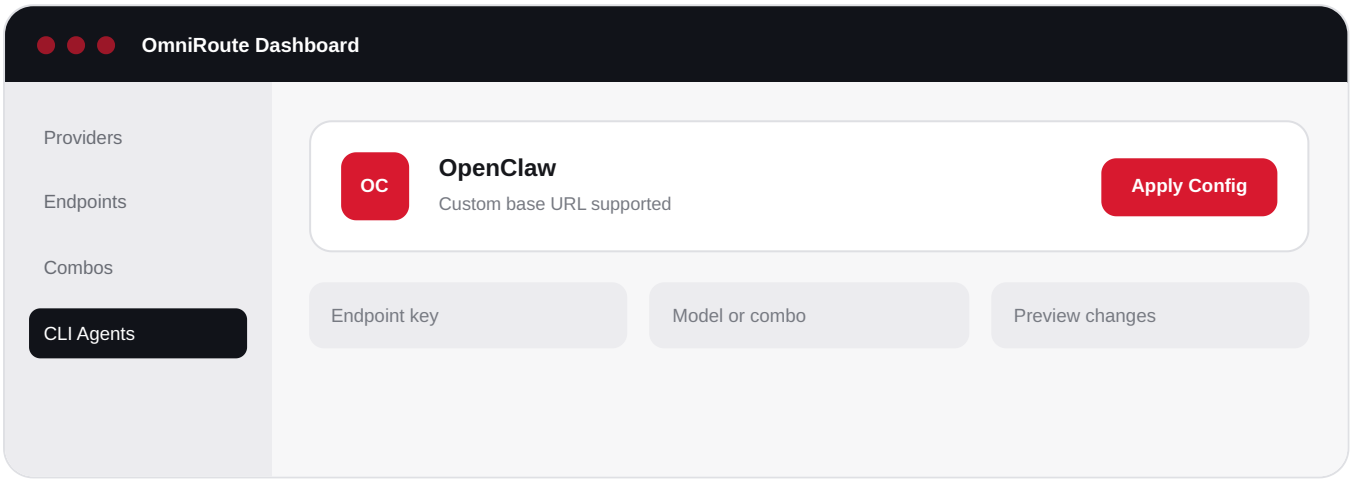
TUTORIAL 3 OF 3 - DASHBOARD

# Configure OpenClaw from OmniRoute

OC

## OmniRoute -> OpenClaw

OmniRoute includes a dedicated OpenClaw integration in its local dashboard, making this the easiest of the two routers for OpenClaw setup.




### 1 Open the CLI Agents page

Go to http://localhost:20128/dashboard/cli-agents and choose **OpenClaw**.

### 2 Choose an endpoint key and model

Select the client-facing OmniRoute key. Use `auto`, a Combo or one exact model ID.

### 3 Preview and apply the configuration

OmniRoute merges its provider into the existing OpenClaw configuration and creates a backup before modifying the file.

### 4 Validate and restart OpenClaw

VERIFY

```
openclaw config validate
openclaw models status
openclaw gateway restart
```

**CONTAINER WARNING**

A setup command run inside the OmniRoute container writes inside that container unless the correct host configuration directories are bind-mounted. Configure host CLIs from the host.

TUTORIAL 3 OF 3 - MANUAL

# Manual OmniRoute Setup for OpenClaw

Use this manual method when you want an environment-backed key, need to audit the exact configuration, or cannot use the dashboard writer.

## 1 Store the endpoint key for the background service

CREDENTIAL FILE

```
mkdir -p ~/.openclaw
cat >> ~/.openclaw/.env <<'EOF'
OMNIROUTE_API_KEY=replace-with-endpoint-key
EOF
chmod 600 ~/.openclaw/.env
```

## 2 Merge the OmniRoute provider

OPENCLAW.JSON - JSON5

```
{
  agents: {
    defaults: {
      model: { primary: "omniroute/auto" }
    }
  },
  models: {
    mode: "merge",
    providers: {
      omniroute: {
        baseUrl: "http://127.0.0.1:20128/v1",
        apiKey: "${OMNIROUTE_API_KEY}",
        api: "openai-completions",
        models: [
          { id: "auto", name: "OmniRoute Auto" }
        ]
      }
    }
  }
}
```

## 3 Validate, select and restart

FINISH

```
openclaw config validate
openclaw models set omniroute/auto
openclaw gateway restart
openclaw models status
```

REMOTE OMNIROUTE

Replace the loopback address with the reachable HTTPS URL of your remote gateway. Do not expose an unauthenticated local router directly to the public internet.

PRACTICAL COMPARISON

# FreeLLMAPI or OmniRoute?

| Decision area | FreeLLMAPI | OmniRoute |
|---|---|---|
| Main purpose | Pool provider free tiers behind one endpoint | Broader routing and provider control plane |
| Default dashboard | `localhost:3001` | `localhost:20128` |
| Claude Code generator | Yes | Yes |
| Codex generator | Yes | Yes |
| OpenClaw experience | Manual custom provider | Dedicated dashboard integration + manual option |
| Routing complexity | Simpler fallback-focused workflow | More advanced combinations, telemetry and controls |
| Best first use | Quick free-tier aggregation | Central gateway for multiple coding tools and agents |

### CHOOSE FREELLMAPI WHEN

- Your first priority is stacking legitimate free-tier access.
- You want a lighter initial setup for Claude Code and Codex.
- You are comfortable adding OpenClaw manually.
- You want one focused router rather than a larger control plane.

### CHOOSE OMNIROUTE WHEN

- You want richer routing, combinations and telemetry.
- You plan to manage several coding tools from one dashboard.
- OpenClaw is a major part of the workflow.
- You are willing to accept a more advanced configuration surface.

### MY PRACTICAL RECOMMENDATION

Start with FreeLLMAPI when the goal is simply “make my free provider access usable.” Start with OmniRoute when the goal is “build a local routing control plane for several tools.” Do not run both just because both exist.

FIX THE BASICS FIRST

# Troubleshooting: URLs and Authentication

**404**

### Not Found or /v1/v1

Check the base URL. Claude uses the root; Codex and OpenClaw use the `/v1` base. A client may append its own path.

**401**

### Unauthorized

Use the FreeLLMAPI unified key or OmniRoute endpoint key - not an upstream provider key. Retest `/v1/models` with the same credential.

**000**

### Connection refused

The router is not running, the port is wrong, or the client is in another network namespace such as Docker.

**ID**

### Unknown model

Fetch the live catalog and copy the exact model ID. Do not guess names from a provider's marketing page.

## Run the diagnosis in this order

1. Router running? -> 2. /v1/models works? -> 3. Client URL correct? -> 4. Model supports tools?

CORE TESTS

```
# FreeLLMAPI
curl http://localhost:3001/v1/models -H "Authorization: Bearer $FREELLMAPI_API_KEY"

# OmniRoute
curl http://localhost:20128/v1/models -H "Authorization: Bearer $OMNIROUTE_API_KEY"
```

**DO NOT DEBUG EVERYTHING AT ONCE**

First prove the router can list models. Then prove one client can send a simple prompt. Only after that should you test tool calling, repository edits and autonomous work.

CLAUDE, CODEX, OPENCLAW AND DOCKER

# Troubleshooting: Client-Specific Problems

### Claude still uses a subscription

Run `/status`. Confirm both the gateway base URL and a gateway credential are active. The URL alone may not replace an existing login.

### Codex ignores the router

Confirm the user config is `~/.codex/config.toml`. Check the selected provider and environment variable in the same shell.

### OpenClaw cannot see the key

Background services may not inherit a terminal export. Place the variable in `~/.openclaw/.env` and restart the gateway.

### OpenClaw config is rejected

Run `openclaw config validate`. Preserve JSON5 structure and merge into existing config rather than pasting a second top-level object.

### Chat succeeds, coding fails

Test file reading, tool calls, command execution, a small edit and the resulting diff. A hello message proves only connectivity.

### Docker localhost confusion

Inside a container, localhost is that container. Use the router service name, a shared network or `host.docker.internal` when supported.

## Useful OpenClaw checks

DIAGNOSTICS

```
openclaw config validate
openclaw models status
openclaw gateway restart
```

## Useful client checks

VERSIONS

```
claude --version
codex --version
openclaw --version
```

A guide cannot fix an outdated or broken client installation. Verify the client before blaming the router.

**REMOTE SETUP RULE**

Use TLS, authentication and network restrictions for any gateway reachable beyond your own machine. A local development convenience should not quietly become a public proxy.

DO NOT SKIP

# Security and Privacy Rules

BT

## Use the convenience. Keep the discipline.

A router centralizes credentials and traffic. That makes it useful - and makes careless configuration more expensive.

### Never publish real keys

- [ ] Replace credentials with placeholders before screenshots.
- [ ] Do not paste .env files into public issues or chat rooms.
- [ ] Rotate any key that appears in a recording or repository.
- [ ] Use endpoint or unified keys only where required.

### Keep secrets out of Git

- [ ] Use environment variables or supported secret stores.
- [ ] Keep OpenClaw daemon credentials in ~/.openclaw/.env.
- [ ] Use Codex env_key rather than a literal key in TOML.
- [ ] Review .gitignore before committing configuration files.

### Know where prompts go

- [ ] Automatic routing may send requests to different providers.
- [ ] Review provider retention and logging policies.
- [ ] Restrict routes for client, medical, financial or government data.
- [ ] Do not assume “local router” means “local inference.”

### Plan for failure

- [ ] Use tested fallbacks, not random model lists.
- [ ] Keep one dependable route for important delivery.
- [ ] Monitor quota and rate-limit behavior.
- [ ] Pin a model when reproducibility matters.

THE LINE WORTH REMEMBERING

**Free is a pricing condition, not an infrastructure strategy.** Use free tiers for learning and experimentation; design reliability deliberately when other people depend on the result.

READY FOR REAL WORK

# Final Setup Checklist

Do not call the setup complete because one “hello” worked. A coding gateway is ready only after the router, client and tool loop have all been tested.

## Router

- [ ] Dashboard opens on the expected local port.
- [ ] At least one provider connection passes its test.
- [ ] A unified or endpoint key has been created.
- [ ] /v1/models returns a non-empty model list.
- [ ] A sensible fallback route is configured.

## Client connection

- [ ] Claude Code shows the correct root URL in /status.
- [ ] Codex selects the router provider from config.toml.
- [ ] OpenClaw config passes validation.
- [ ] The OpenClaw service can read its environment key.

## Capability test

- [ ] A normal text request succeeds.
- [ ] The model can read repository files.
- [ ] A harmless tool call or command succeeds.
- [ ] A small edit can be proposed and applied.
- [ ] The client can explain the resulting diff.

## Safety

- [ ] No real keys appear in screenshots or documentation.
- [ ] Sensitive data is restricted to approved providers.
- [ ] Remote access is authenticated and protected by TLS.
- [ ] A dependable fallback exists for important work.

### Final proof

Ask the client to inspect a small repository, identify one safe improvement, make the change with approval and summarize the diff. That tests more than connectivity.

READY

KEEP THIS PDF BESIDE YOUR TERMINAL

The URL matrix on page 5 and the diagnostic sequence on page 21 solve most setup failures faster than randomly reinstalling everything.

CLICKABLE SOURCES

# Official Links and References

These are the official project and client references used to structure this guide. Open the digital PDF and click any link, or scan the QR code for the BeyondTahir website.

BEYOND / TAHIR

## FreeLLMAPI

**Repository**
https://github.com/tashfeenahmed/freellmapi ->

**Install documentation**
https://github.com/tashfeenahmed/freellmapi/blob/main/docs/install.md ->

**Client integrations**
https://github.com/tashfeenahmed/freellmapi/blob/main/docs/clients/01-agent-clients.md ->

**Setup CLI**
https://github.com/tashfeenahmed/freellmapi/blob/main/cli/README.md ->

## OmniRoute

**Repository**
https://github.com/diegosouzapw/OmniRoute ->

**Setup guide**
https://github.com/diegosouzapw/OmniRoute/blob/release/v3.8.51/docs/guides/SETUP_GUIDE.md ->

**CLI tools guide**
https://github.com/diegosouzapw/OmniRoute/blob/release/v3.8.51/docs/reference/CLI-TOOLS.md ->

**Codex guide**
https://github.com/diegosouzapw/OmniRoute/blob/release/v3.8.51/docs/guides/CODEX-CLI-CONFIGURATION.md ->

## Client documentation

**Claude Code - LLM gateways**
https://docs.anthropic.com/en/docs/claude-code/llm-gateway ->

**Codex - Advanced configuration**
https://developers.openai.com/codex/config-advanced ->

**OpenClaw - Model providers**
https://docs.openclaw.ai/concepts/model-providers ->

**OpenClaw - Installation**
https://docs.openclaw.ai/install ->

## BeyondTahir

**beyondtahir.com**
academy.beyondtahir.com

Practical AI education, implementation and no-code agent building.

**Build it. Test it. Secure it. Then let the agent work.**